

MecaniQA Cloud

Docker, Docker Compose e Kubernetes

Autor: Equipe Macapá

OAT 1 - Infraestrutura containerizada e orquestrada

Agenda

Objetivo

Evidenciar as decisões técnicas, o funcionamento esperado e a cobertura final das demandas.

1. Contexto e arquitetura

Aplicação Java, MySQL e Redis

2. Imagem da API

Build multi-stage e execução segura

3. Orquestração local

Compose, DNS interno e volumes

4. Kubernetes

Deployments, Services, PVCs e probes

5. Evidências e cobertura

K9s, resultados esperados e pacote final

Contexto e arquitetura proposta

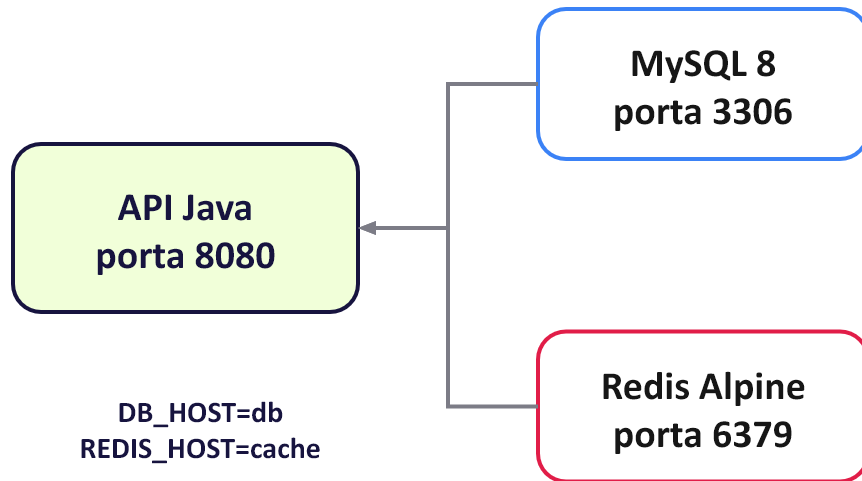
VISÃO GERAL

Problema

Infraestrutura local com pouca capacidade de escala.

Serviços com dependências e ciclos de execução diferentes.

Dados transacionais e cache precisam sobreviver ao reinício dos containers.



Solução implementada

A equipe empacotou a API, conectou os três serviços por nomes DNS e declarou o ambiente Kubernetes.

Mesma arquitetura em dois ambientes

O mecanismo de execução muda. Os nomes de serviço continuam desacoplando a API dos endereços dos containers e Pods.

Imagem Docker da API Java

EMPACOTAMENTO

```
FROM eclipse-temurin:17-jdk-alpine AS build
WORKDIR /app
COPY src ./src
RUN javac --release 17 -d out ...
RUN jar --create --file app.jar ...
```

```
FROM eclipse-temurin:17-jre-alpine
COPY --from=build /app/app.jar ./app.jar
USER mecaniqa
EXPOSE 8080
HEALTHCHECK ... /health
CMD ["java", "-jar", "app.jar"]
```

O que o arquivo garante

Build multi-stage: o JDK compila e o JRE executa o JAR.

Menor privilégio: o processo roda com o usuário mecaniqa.

Health check: o endpoint /health comprova que a API responde.

Limite de memória: a JVM usa até 75% da memória disponível.

Resultado esperado

Imagem na porta 8080 e estado healthy.

Docker Compose: rede e persistência

ORQUESTRAÇÃO LOCAL

```
services:
  db:
    build: docker/mysql
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD:?}
    healthcheck: { test: mysqladmin ping }
    volumes: [mysql_dados:/var/lib/mysql]
  cache:
    build: docker/redis
    healthcheck: { test: redis-cli ping }
    volumes: [redis_dados:/data]
  api:
    build: .
    depends_on:
      db: { condition: service_healthy }
      cache: { condition: service_healthy }
    environment: {DB_HOST: db, REDIS_HOST: cache}
```

Comportamento esperado

Imagens separadas: API, MySQL e Redis possuem Dockerfiles próprios.

Rede dedicada: a API resolve db e cache por DNS interno.

Persistência: mysql_dados e redis_dados preservam o estado fora do ciclo do container.

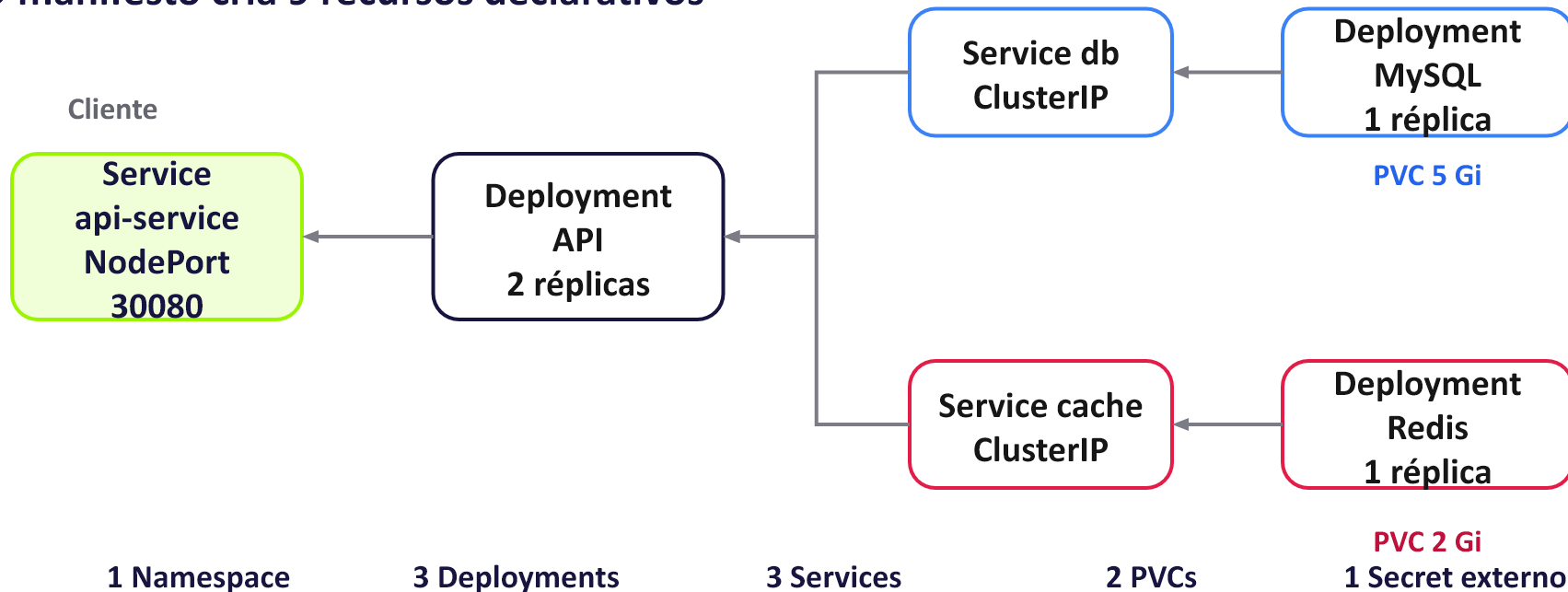
Saúde e segredo: a API aguarda os serviços saudáveis e a senha vem do .env local.

Verificação esperada
docker compose ps + GET /health = serviços saudáveis

Arquitetura declarativa no Kubernetes

NAMESPACE MECANIQA

O manifesto cria 9 recursos declarativos



Saúde, estado e exposição dos serviços

COMPORTAMENTO OPERACIONAL

Probes

```
readinessProbe:  
  httpGet: /health  
livenessProbe:  
  httpGet: /health
```

A prontidão retira Pods indisponíveis do tráfego. A vivacidade permite reinício automático quando a API deixa de responder.

Esperado: 1/1 Ready

Persistência

```
mysql-data: 5Gi  
redis-data: 2Gi  
accessMode:  
  ReadWriteOnce
```

Os PVCs desacoplam os dados do ciclo de vida dos Pods. A recriação do Pod não deve apagar o conteúdo armazenado.

Esperado: PVC Bound

Exposição

```
type: NodePort  
port: 8080  
targetPort: 8080  
nodePort: 30080
```

O Minikube expõe a API pela porta 30080 do nó. MySQL e Redis permanecem acessíveis apenas dentro do cluster.

Esperado: HTTP 200

Evidência de execução no Minikube

CAPTURA DO AMBIENTE



The screenshot shows a VS Code editor with a file explorer on the left containing files like `.dockerignore`, `.gitattributes`, `.gitignore`, `docker-compose.yml`, `Dockerfile`, `kubectl`, `LICENSE`, `logs.txt`, `mecaniga-k8s.yaml`, `minikube-linux-amd64`, and `README.md`. The main editor displays a Kubernetes manifest for a service named `api`, which builds from the local directory and runs a container named `mecaniga-java-api`. The container is configured with port `8080` and depends on a `redis` service.

Below the editor, a terminal window shows the output of `kubectl get pods` for the `mecaniga` namespace. The output shows 4 pods in a `Running` state, all with `READY` status `1/1` and `RESTARTS` of `0`.

NAME	PF	READY	STATUS	RESTARTS	IP	NODE	AGE
api-6cfb5bddb8-56f57	•	1/1	Running	0	10.244.0.7	minikube	9m4s
api-6cfb5bddb8-dfbh8	•	1/1	Running	0	10.244.0.8	minikube	8m53s
mysql-8596bd8b77-rh6mc	•	1/1	Running	0	10.244.0.3	minikube	15m
redis-7c674ccb5b-p9wsw	•	1/1	Running	0	10.244.0.5	minikube	15m

4 Pods em execução: 2 réplicas da API, 1 MySQL e 1 Redis. Todos 1/1 Ready, status Running e 0 reinícios.

Cobertura das demandas da OAT 1

AUDITORIA DO REPOSITÓRIO

Demanda	Implementação encontrada	Status	Resultado esperado
API em container	Dockerfile multi-stage, usuário não root e /health	Atende	Imagem executa na porta 8080
MySQL e Redis	Dockerfiles próprios sobre imagens oficiais	Atende	Serviços independentes e persistentes
Docker Compose	3 serviços, rede, health checks e 2 volumes	Atende	API inicia após banco e cache saudáveis
Kubernetes	3 Deployments, 3 Services, 2 PVCs, probes e limites	Atende	4 Pods Running no Minikube
Monitoramento K9s	Captura com Pods Ready 1/1 e 0 reinícios	Atende	Visibilidade do estado do cluster
Terraform	Provider Kubernetes e 10 recursos equivalentes	Atende	Plan e apply no contexto Minikube
README da equipe	Contexto, arquitetura, comandos e evidências	Atende*	Inserir nomes individuais da equipe

Pacote final e conferência da entrega

FECHAMENTO

Implementação concluída

API Java empacotada com health check e execução sem root.

Compose integra os três Dockerfiles com saúde, DNS e volumes.

Kubernetes usa Secret, probes, PVCs e limites de recursos.

Terraform provisiona a mesma topologia no Minikube.

Antes da entrega

Equipe: substituir o marcador pelos nomes dos integrantes.

Teste: executar Compose e Terraform no computador da apresentação.

Git: versionar o PDF na raiz da branch main.

Blackboard: enviar o mesmo PDF no formulário da disciplina.

Nome do PDF

mecaniQA_oat1_macapa.pdf

Referências

Arquivos do projeto

Dockerfile, docker-compose.yml e
mecanika-k8s.yaml.

terraform/*.tf e README.md do pacote final.

src/br/com/mecanika/Application.java.

Captura do K9s fornecida pela equipe.

OAT 1 - Docker, Docker-compose e K8s.docx.

Documentação oficial

[Dockerfile reference](#) - Docker Docs

[Networking in Compose](#) - Docker Docs

[Deployments](#) - Kubernetes

[Services](#) - Kubernetes

[Persistent Volumes](#) - Kubernetes

[Kubernetes provider](#) - Terraform

Consulta: 23 de setembro de 2026

Obrigado. Perguntas?